

PRACTICAL_WEEK-05

Task 1: Implement a producer-consumer communication system using anonymous pipes, where the parent process acts as the producer and generates data, while the child process acts as the consumer and receives the data. Measure the communication efficiency between the processes.

```
amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
amrutha@Amruthas-MacBook-Air OSSP % pwd
/Users/amrutha/OSSP
amrutha@Amruthas-MacBook-Air OSSP % ls
assets      escape_flowchart.txt  LICENSE      process_flowchart.txt  src          week3_task4_flowchart.txt
bin         File1.txt             Makefile     quote_flowchart.txt    tests
docs        File2.txt             obj          README.md              wait_flowchart.txt
double_quote_flowchart.txt  include            parser_flowchart.txt  scripts               week3_task3_flowchart.txt
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 1: Navigating to the OSSP project directory and viewing the existing project files

```
UW PICO 5.09

}

if (pid > 0)
{
    close(fd[0]);

    gettimeofday(&start, NULL);

    write(fd[1], message, strlen(message) + 1);

    gettimeofday(&end, NULL);

    printf("Parent Process (Producer)\n");
    printf("Sent Data: %s\n", message);

    double time_taken =
        (end.tv_sec - start.tv_sec) * 1000000.0 +
        (end.tv_usec - start.tv_usec);

    printf("Communication Time: %.2f microseconds\n", time_taken);

    close(fd[1]);
}
else
{
    close(fd[1]);

    read(fd[0], buffer, sizeof(buffer));

    printf("Child Process (Consumer)\n");
    printf("Received Data: %s\n", buffer);

    close(fd[0]);
}

return 0;
}
```

Fig 2: C program implementing producer-consumer communication using an anonymous pipe

```

amrutha@Amruthas-MacBook-Air OSSP % gcc src/week5_task1.c -o bin/week5_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/week5_task1
Parent Process (Producer)
Sent Data: Data from Producer to Consumer
Communication Time: 3.00 microseconds
Child Process (Consumer)
Received Data: Data from Producer to Consumer
amrutha@Amruthas-MacBook-Air OSSP % █

```

Fig 3: Compilation and execution of producer-consumer communication using an anonymous pipe

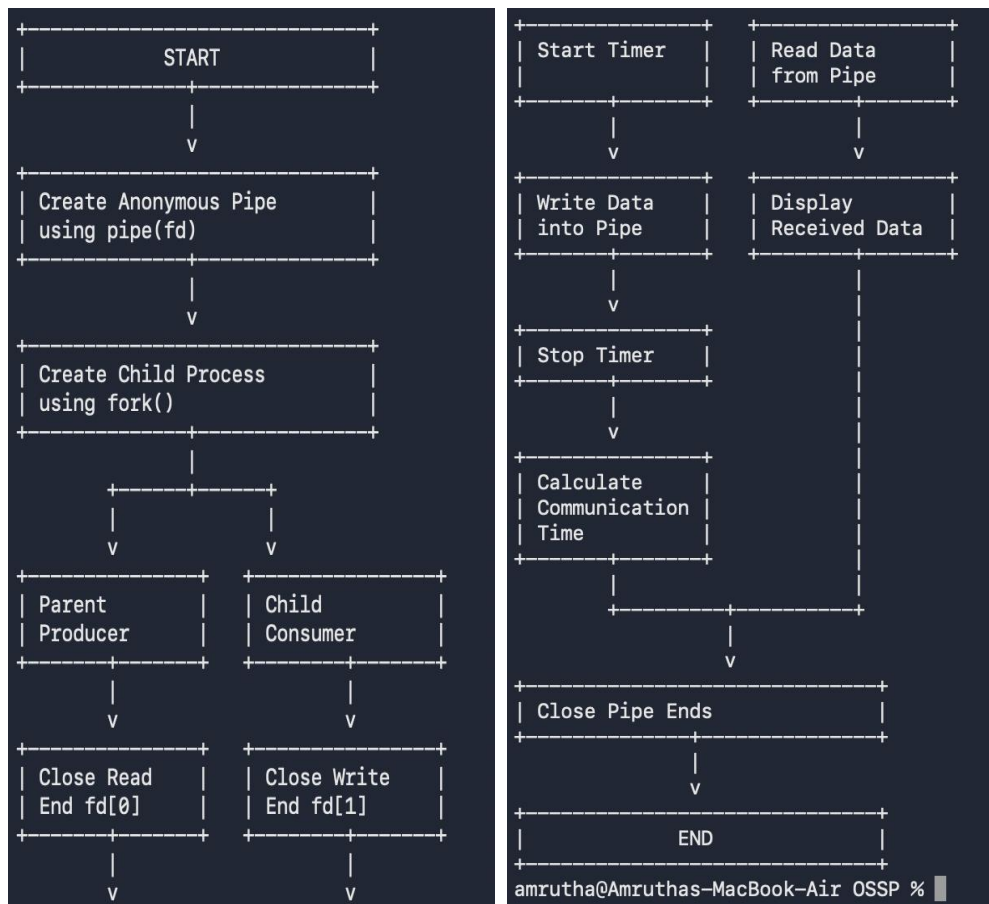


Fig 4: Flowchart of producer-consumer communication using an anonymous pipe

Result:

The producer-consumer system using an anonymous pipe was implemented successfully. The parent process transmitted data through the pipe, and the child process received and displayed the data correctly. The communication time was also measured to observe the efficiency of data transfer between the processes.

Task 2: Implement `ls -l | grep ".c"` Using System Calls

```
UW PICO 5.09

    dup2(fd[1], STDOUT_FILENO);
    close(fd[1]);
    execlp("ls", "ls", "-l", NULL);

    perror("execlp ls failed");
    exit(1);
}

pid2 = fork();
if (pid2 < 0)
{
    perror("fork failed");
    return 1;
}

if (pid2 == 0)
{
    close(fd[1]);
    dup2(fd[0], STDIN_FILENO);
    close(fd[0]);
    execlp("grep", "grep", ".c", NULL);

    perror("execlp grep failed");
    exit(1);
}

close(fd[0]);
close(fd[1]);

wait(NULL);
wait(NULL);

return 0;
}
```

Fig 5: C program implementing `ls -l | grep ".c"` using `pipe()`, `fork()`, `dup2()`, and `exec()`

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/week5_task2.c -o bin/week5_task2
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/week5_task2
drwxr-xr-x  4 amrutha  staff   128 Jul 31 14:05 docs
-rw-r--r--  1 amrutha  staff  1507 Aug 14 19:15 double_quote_flowchart.txt
-rw-r--r--  1 amrutha  staff  1967 Aug 15 10:40 escape_flowchart.txt
drwxr-xr-x  6 amrutha  staff   192 Jul 31 14:05 include
-rw-r--r--  1 amrutha  staff  1177 Aug 14 17:11 parser_flowchart.txt
-rw-r--r--  1 amrutha  staff  1207 Aug 15 15:01 process_flowchart.txt
-rw-r--r--  1 amrutha  staff  1277 Aug 14 17:42 quote_flowchart.txt
drwxr-xr-x  3 amrutha  staff    96 Jul 31 14:05 scripts
drwxr-xr-x 20 amrutha  staff   640 Sep 19 14:14 src
-rw-r--r--  1 amrutha  staff  1702 Aug 17 10:43 wait_flowchart.txt
-rw-r--r--  1 amrutha  staff  1848 Aug 24 20:52 week3_task3_flowchart.txt
-rw-r--r--  1 amrutha  staff  1246 Aug 24 21:20 week3_task4_flowchart.txt
-rw-r--r--  1 amrutha  staff  2036 Sep 19 14:06 week5_task1_flowchart.txt
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 6: Compilation and execution of the program implementing `ls -l | grep ".c"` using system calls



Fig 7: Flowchart for implementing `ls -l | grep ".c"` using `pipe()`, `fork()`, `dup2()`, and `exec()`

Result:

The shell command `ls -l | grep ".c"` was successfully implemented using `pipe()`, `fork()`, `dup2()`, and `exec()` system calls. The output of the `ls -l` command was redirected through an anonymous pipe to the `grep ".c"` process, which filtered and displayed the required `.c` file entries. This demonstrated inter-process communication and command pipeline implementation using Linux system calls.